

Firecracker + VNC GUI - Complete Setup Guide

Goal: Run Chromium browser inside Firecracker microVM with GUI visible on your screen via VNC

Requirements:

- Ubuntu 22.04 LTS or later
 - KVM support (Intel VT-x or AMD-V)
 - 8GB RAM minimum
 - Root/sudo access
-

Part 1: System Preparation

Step 1.1: Verify KVM Support

```
bash

# Check if virtualization is enabled
egrep -c '(vmx|svm)' /proc/cpuinfo
# Should return > 0. If 0, enable in BIOS

# Check if KVM modules are loaded
lsmod | grep kvm
# Should show: kvm_intel or kvm_amd

# If not loaded, load them:
sudo modprobe kvm
sudo modprobe kvm_intel # OR: sudo modprobe kvm_amd
```

Step 1.2: Install Dependencies

```
bash

# Update system
sudo apt update && sudo apt upgrade -y

# Install required packages
sudo apt install -y \
    curl \
    git \
    build-essential \
    qemu-utils \
    debootstrap \
    tigervnc-viewer \
    net-tools

# Verify installation
which curl git debootstrap
```

Part 2: Install Firecracker

Step 2.1: Download Firecracker Binary

```
bash

# Create working directory
mkdir -p ~/firecracker-setup
cd ~/firecracker-setup

# Download latest Firecracker (v1.7.0 as of 2024)
ARCH=$(uname -m)
release_url="https://github.com/firecracker-microvm/firecracker/releases"
latest=$(basename $(curl -fsSLI -o /dev/null -w %{url_effective} ${release_url}/latest))
curl -L ${release_url}/download/${latest}/firecracker-${latest}-${ARCH}.tgz \
    -o firecracker-${latest}-${ARCH}.tgz

# Extract
tar -xzf firecracker-${latest}-${ARCH}.tgz

# Move to system path
sudo mv release-${latest}-${ARCH}/firecracker-${latest}-${ARCH} /usr/local/bin/firecracker
sudo chmod +x /usr/local/bin/firecracker

# Verify installation
firecracker --version

# Should show: Firecracker v1.7.0
```

Step 2.2: Set Up KVM Device Access

```
bash

# Add your user to kvm group
sudo usermod -aG kvm $USER

# Apply group changes (logout/login or run:)
newgrp kvm

# Set KVM device permissions
sudo chmod 666 /dev/kvm

# Verify access
ls -l /dev/kvm
# Should show: crw-rw-rw- ... kvm
```

Part 3: Create Guest OS (Ubuntu Minimal)

Step 3.1: Build Root Filesystem

bash

```
cd ~/firecracker-setup
```

```
# Create directory for rootfs
```

```
mkdir -p rootfs-build
```

```
cd rootfs-build
```

```
# Create minimal Ubuntu rootfs using debootstrap
```

```
sudo debootstrap --arch=amd64 jammy rootfs http://archive.ubuntu.com/ubuntu/
```

```
# Chroot to configure it
```

```
sudo chroot rootfs
```

```
# Inside chroot - configure system
```

```
cat > /etc/hostname << EOF
```

```
firecracker-browser
```

```
EOF
```

```
cat > /etc/network/interfaces << EOF
```

```
auto lo
```

```
iface lo inet loopback
```

```
auto eth0
```

```
iface eth0 inet dhcp
```

```
EOF
```

```
# Set root password
```

```
echo "root:firecracker" | chpasswd
```

```
# Install required packages
```

```
apt update
```

```
apt install -y \
```

```
systemd \
```

```
systemd-sysv \
```

```
udev \
```

```
dbus \
```

```
kmod \
```

```
xserver-xorg \
```

```
xserver-xorg-video-dummy \
```

```
xvfb \
```

```
x11vnc \
```

```
fluxbox \
```

```
chromium-browser \
```

```
net-tools \
```

```
iputils-ping \
```

```
openssh-server
```

```
# Configure automatic services startup
```

```
systemctl enable systemd-networkd  
systemctl enable ssh
```

```
# Exit chroot  
exit
```

Step 3.2: Create Startup Script Inside Rootfs

bash

Create startup script (run this OUTSIDE chroot)

```
sudo tee rootfs/root/start-browser.sh << 'EOF'
```

```
#!/bin/bash
```

```
# Wait for network
```

```
sleep 3
```

```
# Start Xvfb (virtual display)
```

```
export DISPLAY=:0
```

```
Xvfb :0 -screen 0 1920x1080x24 &
```

```
sleep 2
```

```
# Start window manager
```

```
fluxbox &
```

```
sleep 1
```

```
# Start VNC server
```

```
x11vnc -display :0 -forever -shared -nopw -rfbport 5900 &
```

```
sleep 2
```

```
# Start Chromium
```

```
chromium-browser \
```

```
    --no-sandbox \
```

```
    --disable-dev-shm-usage \
```

```
    --start-maximized \
```

```
    --disable-gpu \
```

```
    --disable-software-rasterizer &
```

```
# Keep script running
```

```
tail -f /dev/null
```

```
EOF
```

```
sudo chmod +x rootfs/root/start-browser.sh
```

```
# Create systemd service for auto-start
```

```
sudo tee rootfs/etc/systemd/system/browser-isolation.service << 'EOF'
```

```
[Unit]
```

```
Description=Browser Isolation GUI
```

```
After=network.target
```

```
[Service]
```

```
Type=simple
```

```
User=root
```

```
ExecStart=/root/start-browser.sh
```

```
Restart=on-failure
```

```
[Install]
```

```
WantedBy=multi-user.target
```

```
EOF
```

```
# Enable the service
```

```
sudo chroot rootfs systemctl enable browser-isolation.service
```

Step 3.3: Create ext4 Filesystem Image

```
bash
```

```
# Exit rootfs directory
```

```
cd ~/firecracker-setup/rootfs-build
```

```
# Calculate required size (rootfs size + 2GB extra)
```

```
ROOTFS_SIZE=$(sudo du -sm rootfs | cut -f1)
```

```
IMAGE_SIZE=$((ROOTFS_SIZE + 2048))
```

```
# Create empty file
```

```
dd if=/dev/zero of=../rootfs.ext4 bs=1M count=$IMAGE_SIZE
```

```
# Format as ext4
```

```
mkfs.ext4 ../rootfs.ext4
```

```
# Mount and copy files
```

```
mkdir -p /tmp/rootfs-mount
```

```
sudo mount ../rootfs.ext4 /tmp/rootfs-mount
```

```
sudo cp -a rootfs/* /tmp/rootfs-mount/
```

```
sudo umount /tmp/rootfs-mount
```

```
echo "✅ Rootfs created: ~/firecracker-setup/rootfs.ext4"
```

Part 4: Create Linux Kernel

Step 4.1: Download Pre-built Kernel (Easiest)

```
bash
```

```
cd ~/firecracker-setup
```

```
# Download Firecracker-compatible kernel
```

```
curl -fsSL -o vmlinux.bin \  
https://s3.amazonaws.com/spec.ccfc.min/img/quickstart_guide/x86_64/kernels/vmlinux
```

```
echo "✅ Kernel downloaded: ~/firecracker-setup/vmlinux.bin"
```

OR Build Custom Kernel (Advanced):

```
bash
```

```
cd ~/firecracker-setup
```

```
# Download kernel source
```

```
wget https://cdn.kernel.org/pub/linux/kernel/v5.x/linux-5.10.186.tar.xz
```

```
tar -xf linux-5.10.186.tar.xz
```

```
cd linux-5.10.186
```

```
# Get Firecracker kernel config
```

```
curl -fsSL -o .config \  
https://raw.githubusercontent.com/firecracker-microvm/firecracker/main/resources/g
```

```
# Build kernel
```

```
make vmlinux -j$(nproc)
```

```
# Copy to setup directory
```

```
cp vmlinux ../vmlinux.bin
```

```
cd ..
```

Part 5: Configure Firecracker

Step 5.1: Create VM Configuration File


```
bash
```

```
cd ~/firecracker-setup
```

```
# Create config.json
```

```
cat > config.json << 'EOF'
```

```
{
  "boot-source": {
    "kernel_image_path": "/home/YOUR_USERNAME/firecracker-setup/vmlinux.bin",
    "boot_args": "console=ttyS0 reboot=k panic=1 pci=off root=/dev/vda rw"
  },
  "drives": [
    {
      "drive_id": "rootfs",
      "path_on_host": "/home/YOUR_USERNAME/firecracker-setup/rootfs.ext4",
      "is_root_device": true,
      "is_read_only": false
    }
  ],
  "machine-config": {
    "vcpu_count": 2,
    "mem_size_mib": 2048,
    "ht_enabled": false
  },
  "network-interfaces": [
    {
      "iface_id": "eth0",
      "guest_mac": "AA:FC:00:00:00:01",
      "host_dev_name": "tap0"
    }
  ]
}
EOF
```

```
# Replace YOUR_USERNAME with actual username
```

```
sed -i "s|YOUR_USERNAME|$USER|g" config.json
```

```
echo "✅ Configuration created: ~/firecracker-setup/config.json"
```

Step 5.2: Set Up Network (TAP Device)

```
bash
```

```
# Create network setup script
```

```
cat > setup-network.sh << 'EOF'
```

```
#!/bin/bash
```

```
# Create TAP device
```

```
sudo ip tuntap add tap0 mode tap
```

```
sudo ip addr add 172.16.0.1/24 dev tap0
```

```
sudo ip link set tap0 up
```

```
# Enable IP forwarding
```

```
sudo sh -c "echo 1 > /proc/sys/net/ipv4/ip_forward"
```

```
# Set up NAT for internet access
```

```
sudo iptables -t nat -A POSTROUTING -o $(ip route | grep default | awk '{print $5}')
```

```
sudo iptables -A FORWARD -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
```

```
sudo iptables -A FORWARD -i tap0 -o $(ip route | grep default | awk '{print $5}') -j
```

```
echo "✅ Network configured: tap0 at 172.16.0.1"
```

```
EOF
```

```
chmod +x setup-network.sh
```

```
# Run it
```

```
./setup-network.sh
```

Part 6: Launch Firecracker VM

Step 6.1: Start Firecracker

```
bash
```

```
cd ~/firecracker-setup
```

```
# Remove old socket if exists
```

```
sudo rm -f /tmp/firecracker.socket
```

```
# Start Firecracker in background
```

```
firecracker --api-sock /tmp/firecracker.socket &
```

```
# Wait for socket creation
```

```
sleep 2
```

```
# Apply configuration
```

```
curl --unix-socket /tmp/firecracker.socket -X PUT \  
-H "Content-Type: application/json" \  
-d @config.json \  
http://localhost/boot-source
```

```
curl --unix-socket /tmp/firecracker.socket -X PUT \  
-H "Content-Type: application/json" \  
-d @config.json \  
http://localhost/drives/rootfs
```

```
curl --unix-socket /tmp/firecracker.socket -X PUT \  
-H "Content-Type: application/json" \  
-d @config.json \  
http://localhost/machine-config
```

```
curl --unix-socket /tmp/firecracker.socket -X PUT \  
-H "Content-Type: application/json" \  
-d @config.json \  
http://localhost/network-interfaces/eth0
```

```
# Start the VM
```

```
curl --unix-socket /tmp/firecracker.socket -X PUT \  
-H "Content-Type: application/json" \  
-d '{"action_type": "InstanceStart"}' \  
http://localhost/actions
```

```
echo "✅ Firecracker VM starting..."
```

Step 6.2: Create Launcher Script (For Easy Use)

bash

```
cat > launch-firecracker-browser.sh << 'EOF'
#!/bin/bash

echo "🚀 Starting Firecracker Browser Isolation..."

# Setup network
./setup-network.sh

# Clean up old socket
sudo rm -f /tmp/firecracker.socket

# Start Firecracker
firecracker --api-sock /tmp/firecracker.socket \
  --config-file config.json &

FIRECRACKER_PID=$!

# Wait for VM to boot
echo "⌚ Waiting for VM to boot (30 seconds)..."
sleep 30

# Set up port forwarding for VNC
# VM will have IP 172.16.0.2 (DHCP from tap0)
# Forward localhost:5900 to VM:5900
sudo iptables -t nat -A PREROUTING -p tcp --dport 5900 -j DNAT --to-destination 172.16.0.2
sudo iptables -A FORWARD -p tcp -d 172.16.0.2 --dport 5900 -j ACCEPT

# Open VNC viewer
echo "🌐 Opening VNC connection..."
vncviewer localhost:5900 &

echo "✅ Browser isolation ready!"
echo "    - Close VNC viewer to stop"
echo "    - Press Ctrl+C to terminate Firecracker"

# Wait for Firecracker
wait $FIRECRACKER_PID
EOF

chmod +x launch-firecracker-browser.sh
```

Part 7: Connect to Browser

Step 7.1: Test VNC Connection

```
bash

# VM should be running now
# Wait 30-60 seconds for full boot

# Check if VM is accessible
ping -c 3 172.16.0.2

# Connect with VNC viewer
vncviewer 172.16.0.2:5900
# OR
vncviewer localhost:5900 # if port forwarding is set up

# You should see Chromium browser!
```

Step 7.2: Troubleshooting

If browser doesn't appear:

```
bash

# SSH into the VM to debug
ssh root@172.16.0.2
# Password: firecracker

# Check if services are running
systemctl status browser-isolation
ps aux | grep Xvfb
ps aux | grep x11vnc
ps aux | grep chromium

# Manually start if needed
/root/start-browser.sh

# Check VNC is listening
netstat -tulpn | grep 5900

# View logs
journalctl -u browser-isolation -f
```

Part 8: Create Desktop Launcher

Step 8.1: Create Desktop Entry

```
bash
```

```
cat > ~/.local/share/applications/firecracker-browser.desktop << EOF
[Desktop Entry]
Version=1.0
Type=Application
Name=Isolated Browser (Firecracker)
Comment=Launch browser in isolated Firecracker microVM
Exec=$HOME/firecracker-setup/launch-firecracker-browser.sh
Icon=web-browser
Terminal=false
Categories=Network;WebBrowser;Security;
EOF

# Update desktop database
update-desktop-database ~/.local/share/applications/

echo "✅ Desktop launcher created"
echo "    Look for 'Isolated Browser (Firecracker)' in applications menu"
```

Part 9: File Monitoring Setup

Step 9.1: Create Monitor Script (Runs on Host)

bash

```
cat > ~/firecracker-setup/file-monitor.py << 'EOF'
#!/usr/bin/env python3
"""
Monitor file operations in Firecracker VM
"""

import subprocess
import time
import logging

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s'
)

def monitor_vm_files():
    """Monitor files in VM via SSH"""
    vm_ip = "172.16.0.2"
    watch_path = "/root/Downloads"

    logging.info(f"👁 Monitoring VM:{vm_ip} path:{watch_path}")

    # SSH into VM and watch directory
    ssh_cmd = f"""
ssh -o StrictHostKeyChecking=no root@{vm_ip} '
inotifywait -m {watch_path} -e create -e modify -e delete
'
"""

    proc = subprocess.Popen(
        ssh_cmd,
        shell=True,
        stdout=subprocess.PIPE,
        stderr=subprocess.PIPE,
        text=True
    )

    try:
        for line in proc.stdout:
            logging.warning(f"🚨 FILE EVENT: {line.strip()}")
    except KeyboardInterrupt:
        logging.info("Monitoring stopped")
        proc.kill()

if __name__ == "__main__":
    monitor_vm_files()
EOF
```

```
chmod +x ~/firecracker-setup/file-monitor.py
```

Part 10: Performance Testing

Step 10.1: Benchmark Boot Time

```
bash

cat > ~/firecracker-setup/benchmark-boot.sh << 'EOF'
#!/bin/bash

echo "🕒 Benchmarking Firecracker boot time..."

# Clean start
sudo rm -f /tmp/firecracker.socket
./setup-network.sh

# Measure boot time
START_TIME=$(date +%s%3N)

firecracker --api-sock /tmp/firecracker.socket \
  --config-file config.json &

FIRECRACKER_PID=$!

# Wait for VM to be responsive
while ! ping -c 1 172.16.0.2 &> /dev/null; do
  sleep 0.1
done

END_TIME=$(date +%s%3N)
BOOT_TIME=$((END_TIME - START_TIME))

echo "✅ Firecracker microVM boot time: ${BOOT_TIME}ms"

# Cleanup
kill $FIRECRACKER_PID
EOF

chmod +x ~/firecracker-setup/benchmark-boot.sh
```

Configuration Files Summary

All files are in: `~/firecracker-setup/`


```
firecracker-setup/  
├─ vmlinux.bin           # Linux kernel  
├─ rootfs.ext4           # Root filesystem  
├─ config.json           # Firecracker config  
├─ setup-network.sh      # Network setup  
├─ launch-firecracker-browser.sh # Main launcher  
├─ file-monitor.py       # File monitoring  
└─ benchmark-boot.sh     # Performance test
```

Quick Start Commands

```
bash  
  
# 1. Setup network  
cd ~/firecracker-setup  
./setup-network.sh  
  
# 2. Launch browser  
./launch-firecracker-browser.sh  
  
# 3. Monitor files (in another terminal)  
python3 file-monitor.py  
  
# 4. Benchmark performance  
./benchmark-boot.sh
```

Expected Performance

- **VM Boot Time:** 125-200ms
 - **Total Start to Browser:** 3-5 seconds
 - **Memory Usage:** ~2GB RAM per session
 - **Display Latency:** 50-100ms (VNC overhead)
-

Advantages & Disadvantages

✅ **Advantages:**

- Strongest isolation (own kernel)
- Minimal attack surface
- AWS-proven technology
- Sub-200ms boot time

✗ Disadvantages:

- Complex VNC setup
 - No GPU acceleration
 - Linux host only
 - Steeper learning curve
-

Next Steps

1. Test basic isolation
2. Add YARA malware scanning
3. Implement policy enforcement
4. Create user-friendly GUI wrapper
5. Add session logging

This completes the Firecracker setup! 🎉